

AI Image Understanding & Content Matching Engine

Understand an image library, organize it automatically, and match the right image to the right article — a red-fox post gets the red-fox photo, never the wolf. Good suggestions when confident, safe rejection when not.

Difficulty: Medium

Self-paced · no deadlines

JavaScript or Python

Public GitHub repo

\$0 · no credit card, ever

THE FLAVOR

Vision AI + semantic matching, with a mismatch guard — production AI reliability

YOU WILL MASTER

Structured vision output · embeddings · similarity thresholds · eval-driven quality

YOUR \$0 STACK

Node or Python · Gemini Flash free tier or local Ollama · ~50 free images

How to read this document: § 1–2 tell you what this capstone is and whether it's the right pick for you. § 3–9 are the build: rules, features, architecture, the definition of done, and the build phases. § 10–12 are the practical frame: the free tools, the GitHub rules, and exactly how you'll be evaluated. § 13–14 prepare your demo and give you hand-picked, free resources for every phase. You don't need to memorize it — work through the § 8 phases at your own pace and come back when you need detail.

Contents

1. The mission.....	1	9. Stretch goals	6
2. What it takes to finish.....	2	10. Your \$0 stack	6
3. Ground rules	3	11. GitHub rules.....	8
4. What you'll build.....	3	12. How it's evaluated	9
5. Architecture overview	4	13. The final demo	10
6. Definition of done	4	14. Curated resources.....	11
7. Realistic scope	5	15. Glossary	12
8. The build, phase by phase	5		

1 • The mission

Build a system that looks at an image library, **understands what's actually in each image**, tags it, and matches each image to the right blog post — based on **what the images mean**, not filenames or keywords.

The behavior you're aiming for:

- A blog post about **red foxes** surfaces a red-fox image.
- A similar-looking **wolf** image is rejected.
- A generic **dog** image ranks poorly.

- And if no image is a good enough match, **the system says so instead of guessing.**

That last line is the whole point. The most important production feature is not finding a match — it is **avoiding a wrong match**. You'll build a **mismatch guard**: a safety layer that combines extracted tags, semantic similarity thresholds, and confidence scores to decide *"is this recommendation actually good enough?"* — and refuses with an explanation when it isn't.

This is the reliability expected from real AI systems: **good suggestions when confident, safe rejection when uncertain**. You are not building an image search engine; you are building a trustworthy AI decision system — the difference between a demo and a product.

The friendliest AI capstone. Rated Medium, with the most bounded AI scope: ~50 images, a few categories, a small labeled eval set. Do the Challenge 2 relevance-gate assignment first — it's the decision core of this capstone in one sitting; the capstone builds the whole system around it.

2 · What it takes to finish

Honest picture before you commit — **Medium**, but with real teeth in three places:

The three genuinely hard parts

- **Trusting the model just enough.** Vision output must be schema-validated, and low-confidence classifications get flagged, not accepted. Learning to treat an AI as an unreliable-but-useful component is the core lesson.
- **Tuning the guard.** Where exactly is "similar enough"? You'll pick thresholds using your own labeled eval set — and defend the fox/wolf boundary at the demo with a precision number, not a feeling.
- **Batch discipline.** Vision calls are slow and bulk — they run as background jobs with retries and per-call **cost tracking**, even inside a free tier. Habits, not heroics.

Time budget: roughly **35–50 focused hours**, at your own pace. The pipeline is steady work; keep a block in Phase 4 for the eval set — the precision number is your demo's closing argument.

You already have the parts

Piece of this capstone	Where you already built it
Structured vision output with schema validation	A11
AI cost tracking	A12
Embeddings & retrieval	A15
Background batch processing	A9
The matching + relevance-guard core	Challenge 2

Pick this one if you want AI depth with the most contained scope — and a demo moment ("watch it refuse the wolf") that lands with any audience, technical or not.

3 · Ground rules — read before you start

These rules are the same for every capstone in the track. They exist so that 20,000+ interns can be evaluated fairly — and so your finished project is something you can safely show in an interview.

The five rules

Rule	What it means for you
Pick one, early	This internship is self-paced — no deadlines. Still, choose your capstone early (once the first assignments have shown you the landscape) and write a one-page design doc (problem, data model, API surface, layer sketch, one explicit non-goal). Phase 1 in § 8 is exactly that doc.
One separate, public repo	The capstone lives in its own public GitHub repository from day one — never inside your assignments repo. Full rules in § 11.
\$0, no credit card — ever	Everything can be built with free tools; this document lists the exact free stack in § 10. If you ever find yourself on a page asking for a credit card, stop — you took a wrong turn, the free path exists.
AI-assisted building is encouraged — and owned	Use AI tools freely, but keep <code>BUILDLLOG.md</code> honest: where AI helped, where it was wrong, what you changed. At the demo you will be asked to explain 2–3 lines the evaluator picks. <i>"The AI wrote it"</i> is not an answer.
Propose your own	You may pitch a different capstone idea against the shared requirements (§ 12). If it exercises a comparable skill set, mentors can approve it.

CONSTRAINTS FOR THIS CAPSTONE

- **Stay in the free tier:** Gemini Flash's free tier (Google account, no card) or fully local models via Ollama. ~50 images keeps every run cheap; batch jobs + cost tracking keep it visible.
- **Never trust invalid model output.** Every vision response is validated against your schema; failures are retried or flagged — never silently accepted.
- **Use licensed-free images** for your corpus (Unsplash / Pexels — licenses linked in § 14). Keep the corpus small and commit it (or a download script) so evaluators can reproduce your results.
- **API keys live in `.env`**, never in code or commits.

4 · What you'll build

Given a set of images and a set of posts, you build a service with five parts:

1 Image ingestion & classification

teaches: structured AI output

Run every image through a vision model and produce **validated, structured metadata**:

```
{
  "subject": "red fox",
  "category": "animal",
  "attributes": ["orange fur", "wild", "forest"],
  "caption": "A red fox standing in a forest",
  "confidence": 0.94
}
```

Validate against a schema. Never trust invalid responses. **Low-confidence results get flagged instead of guessed.**

2 Semantic image matching

teaches: semantic matching

Create **embeddings** for image descriptions *and* blog post content, store them in a vector index, and for each post rank the most relevant images. The system must match **concepts, not exact words** — "red fox", "Vulpes vulpes", and "wild fox species" are related even though the words differ.

3 The mismatch guard

teaches: AI safety layers

The production-critical part: a safety layer that decides "*is this recommendation actually good enough?*" by combining tag validation, semantic similarity thresholds, and confidence scores:

```
Post:      "The behavior of red foxes"
Candidate: "A gray wolf in the forest"
Result:    REJECTED
Reason:    "Animal category mismatch: expected fox, detected wolf"
```

Knowing when the *best* candidate is still *wrong* — and explaining why — is what separates production AI from demos.

4 Background processing system

teaches: batch jobs + cost control

Vision and embedding generation run asynchronously as **batch jobs** with retries, progress tracking, and **per-call cost tracking** — model it on FlyRank's batch-classification pattern. Slow, bulk AI work never blocks a request.

5 Review API

teaches: human-in-the-loop review

A simple workflow to approve or reject a suggested pairing and inspect *why* an image was selected or refused. A full UI is not required — validated endpoints plus a table are enough.

5 · Architecture overview

Two parallel embedding streams meet at the ranking step, and everything passes the guard before a human sees it:

```
Images -(batch job)→ Vision Model → {tags, caption, confidence} → image_metadata
                                     ↳ embed(caption) → image_vectors

Posts → embed(post text) → post_vectors

GET /posts/:id/images
  ↳ Similarity Ranking (image_vectors × post_vector)
    ↳ Mismatch Guard (tags + threshold + confidence)
      ↳ Suggested image (ranked, explained)
      ↳ "No good match" + explanation
        ↳ Review API: approve / reject
```

6 · Definition of done — the core checklist

This is the contract. **Done** = every box below ticked, with one pasted proof per box in **EVIDENCE.md**. Each box is written so a reviewer can verify it in minutes.

AI PROCESSING

- ☐ Vision model produces structured output validated against a schema; invalid responses are never trusted.
- ☐ Low-confidence classifications are flagged instead of accepted.
- ☐ Images are processed through a batch background job with retries.
- ☐ Vision and embedding costs are tracked per call.

MATCHING SYSTEM

- ☐ Image and post embeddings are stored; posts return ranked image suggestions.
- ☐ Semantic matching works for equivalent concepts — "red fox" matches "Vulpes vulpes".

SAFETY LAYER

- ☐ The mismatch guard rejects incorrect recommendations — the wolf-on-a-fox-post scenario provably fails.
- ☐ Rejections include a human-readable explanation.
- ☐ When no image clears the bar, the system answers "no confident match" with reasons.

BACKEND

- ☐ Database models for images, tags, embeddings, posts, suggestions, approvals/rejections — with the required indexes.
- ☐ API endpoints validated; the review workflow (approve / reject / inspect why) exists.

QUALITY & DOCUMENTATION

- ☐ Automated tests cover schema validation, mismatch rejection, and matching accuracy.
- ☐ A small labeled evaluation dataset measures top-1 precision — the number is in your README.
- ☐ README with architecture explanation and diagram; submission-pack files from § 11 present.

7 · Realistic scope — where to stop

Do not build a massive image platform. The right size:

- **~50 images across a few categories** — e.g. animals: red fox, wolf, dog, bear, deer. Large enough to show real retrieval behavior, small enough to process cheaply and check by eye.
- **The review interface can be API endpoints, a simple admin table, or a minimal internal page.** No frontend build.
- **Reuse the Challenge 2 eval set** as your test seed and grow it slightly.
- **One vision model + one embedding model is enough.** Comparing models is a stretch, not core.

8 · The build, phase by phase

This internship is **self-paced** — there is no calendar and no deadline. Work through the phases **in order, at your own speed**: the track assignments are the parts, the capstone assembles them. Each phase ends with a **gate** — a concrete result that tells you it's safe to move on. The effort estimates are just orientation; take what you need. Short on time overall? Shrink scope (§ 7), don't skip phases.

Phase 1 Design · ≈4–6 h

gate: design doc signed off

- Image metadata schema (the tag JSON above)
- Matching strategy + guard rules sketched
- Database design
- Initial ~50-image dataset gathered

Phase 2 Image understanding pipeline · ≈10–14 h

gate: all images tagged by the batch job, costs visible

- Vision processing job with structured output validation
- Batch processing with retries
- Cost tracking per call

Phase 3 Matching engine · ≈12–16 h

gate: fox post ranks fox first; guard refuses the wolf

- Embeddings for images + posts
- Similarity search + ranking
- The mismatch guard with explanations

Phase 4 Production layer · ≈8–12 h

gate: eval precision measured; tests green

- Review API · evaluation dataset · automated tests
- README + architecture diagram + `EVIDENCE.md` filled as you go

Phase 5 Demo prep · ≈2–3 h

gate: § 13 rehearsed twice, forced-wolf moment ready

- Seed the corpus, rehearse the guard refusal and the "no confident match" case

9 · Stretch goals — only if the core ships



Optional, and only after every § 6 box is green. A finished core with one polished stretch beats three half-stretches. Each of these is a genuine "I went deep" interview story:

- **Automatic alt text** generated from image understanding (a real FlyRank concern).
- **Near-duplicate detection** using perceptual hashes or embedding distance.
- **Fallback image generation** when no suitable image exists (ties to `generateImageForContent.ts`).
- **Human-in-the-loop agent QA** for uncertain matches.
- **Run it as a service node** inside the Streams platform (Capstone 1 family).

10 · Your \$0 stack — the free-tools promise

We promised you can finish this internship without paying for anything. Here is the proof for this capstone. Every requirement maps to a tool that is free with **no credit card**:

You need	Free tool (no credit card)	Notes
----------	----------------------------	-------

Language + framework	Node.js + Express or Python + FastAPI	Free, as all track long
Vision model (cloud path)	Gemini Flash — image understanding, free tier	Google account + AI Studio key; no card
Vision model (local path)	Ollama vision models (e.g. llama, moondream)	Fully local, no key, works offline
Embeddings	Gemini embeddings free tier, or Ollama <code>all-minilm</code>	Free either way
Schema validation	Zod · Pydantic	Free libraries
Database	PostgreSQL via Docker (pgvector optional at ~50 images)	Free · in-DB arrays fine at this scale
Image corpus	Unsplash / Pexels (licenses in § 14)	Free to use; check the license pages
Repo	GitHub (public)	Free

The iron rule: if any tool, tier, or tutorial asks for a credit card, it is the wrong path — a free alternative for this capstone exists in the table above. Stuck anyway? Ask in the community before paying for *anything*.

11 · GitHub rules — your public repo

Your capstone is also your **portfolio piece**. These rules make the repo something a recruiter, a mentor, and our automated evaluator can all navigate without asking you anything.

The non-negotiables

- ☐ **One dedicated repository, public from day one.** Create it the day you choose your capstone. Do **not** build in your assignments repo, a private repo you flip later, or a monorepo shared with other work. Progress in the open is part of the story — an honest half-built history beats a single "final version" push.
- ☐ **Name it clearly:** `flyrank-capstone-<short-name>` (for this capstone we suggest `flyrank-capstone-image-relevance`). Lowercase, hyphens, no spaces.
- ☐ **Commit as you build.** Small, meaningful commits with messages that say what changed (`Add idempotency key check to publish endpoint`, not `update stuff`). Aim for at least one commit per working session; each phase in § 8 should be visible in your history.
- ☐ **Never commit a secret.** No API keys, tokens, passwords, or `.env` files — put `.env` in `.gitignore` before your first commit and ship a `.env.example` with placeholder values instead. A leaked key in your history means rotating the key and rewriting history — ask for help the moment it happens.
- ☐ **A stranger can run it.** The README's setup section must work on a clean machine with one documented run command (`docker compose up` or equivalent) plus a seed step for demo data.

Required files at submission

File	What goes in it
<code>README.md</code>	What the system does, an architecture diagram (an image or ASCII sketch), exact run + seed steps, and an honest "limitations" note.
<code>capstone.yaml</code>	A small manifest the evaluator reads: <code>run:</code> (one command), <code>seed:</code> , <code>test:</code> , <code>base_url:</code> and the endpoints to probe.
<code>EVIDENCE.md</code>	One pasted proof per Definition-of-Done checkbox in § 6 — a test name + output, a curl transcript, or a log line. Claims without evidence score as not done.
<code>BUILDLOG.md</code>	Your AI-usage log: where AI helped, where it was wrong, what you changed. Honesty is graded, perfection is not.
<code>.env.example</code>	Every environment variable the app needs, with safe placeholder values.

DO

- Create the repo public before you build, first commit = README skeleton + `.gitignore`.
- Add a license (MIT is a fine default) so others may read and learn from it.
- Use branches if you like — but keep `main` always runnable.
- Paste real command output into `EVIDENCE.md` as you finish each checkbox — not in a panic at the end.

DON'T

- Don't mix capstone code with track assignments — separate repos, always.
- Don't commit `node_modules/`, virtualenvs, or datasets over a few MB.
- Don't force-push over your history before submission — the journey is evidence.
- Don't commit real tokens "just for a second". There is no such thing on a public repo.

12 · How your capstone is evaluated

Three layers, published up front — **you know exactly what will be checked, so build to pass it.**

Layer 1 — The submission pack (machine-checkable)

The portal first checks your repo structure: the five required files from § 11, a `run:` command that boots the system, and a `test:` command that runs green. Missing pack files cost points before a human ever looks.

Layer 2 — Acceptance probes (behavioral, pass/fail)

An evaluator (human or automated) runs these against your live system. They are not secrets — they are promises:

PROBE 1 — Run the batch job on the corpus → every image gains schema-valid tags; at least one low-confidence image is flagged, not guessed.

PROBE 2 — Query images for the "red fox" article → the fox image ranks first; wolf and dog rank clearly lower.

PROBE 3 — Force the wolf as a candidate for the fox post → the guard rejects it with a category-mismatch explanation.

PROBE 4 — Query a post with no suitable image → "no confident match" + reasons (similarity below threshold / subjects don't match).

PROBE 5 — Run the eval script → top-1 precision reported on the labeled set, matching the number in the README.

PROBE 6 — Check the cost log → every vision/embedding call attributed with a cost entry.

Layer 3 — The rubric (judgment, 1–5 per dimension)

The weights say what we value: **correctness and resilience over feature count.**

Dimension	What a "5" looks like	Weight
Architecture	Clean layers; swap the DB or a provider without touching business logic	×3
Correctness	Happy path solid; edge cases handled; probes all pass	×3
Resilience	Survives bad input, a dead dependency, a job running twice	×3
Security	Real authorization; validated input; secrets encrypted & never logged	×2
AI cost & grounding (if AI used)	Cost metered + capped; outputs grounded; eval catches regressions	×2
Testing	The scary cases, deterministically; evals for AI behavior	×2
Communication	README + diagram tell the story; honest about limits; demo lands	×2

Levels: **Ships** = every core box in § 6 done + all probes pass (a real pass) · **Solid** = Ships + rubric average ≥ 3.5 · **Exceptional** = Solid + meaningful stretch goals. A small system that is correct, resilient and well-tested beats a huge one that falls over — that is what senior engineers actually value.

The shared requirements (every capstone must show these)

These eight patterns are how we know you absorbed the program, not just this topic. You built each one in an assignment already:

#	Requirement	Where you learned it
1	Layered architecture — data / logic / HTTP separated	A2 + the architecture live event
2	Validation at the boundary — bad input → clean 4xx, never a 500	A5
3	≥1 background job — slow/bulk work off the request path, retries + failure alert	A7
4	Real persistence — schema as migrations, right indexes, isolated tenants	A5
5	Idempotency where it matters — the retried action happens once	A5 stretch + A10
6	Secrets clean — env only, encrypted if stored, never logged	A10
7	Cost tracked, if AI is used — per call, attributed, with a budget guard	A6
8	Tests that matter — the scary cases, deterministic; AI features get an eval	A13

13 · The final demo — your 6 minutes

Rehearse this flow start to finish. A demo that shows **one failure handled gracefully** impresses more than ten happy paths.

1. Show a folder of animal photos being processed — tags and captions appearing automatically.
2. Open the "red fox" article → the fox photo surfaces on top; wolf and dog rank far below.
3. **Force the wolf recommendation** → the mismatch guard refuses it, with its reason on screen.
4. Show a post where nothing fits → "No confident match found. Similarity below threshold; detected subjects do not match article topic."
5. Approve one suggestion, reject another — the review trail records both.
6. Close with your eval number: *"Top-1 precision: 92%"* (yours will vary — what matters is that you measured it).
7. Final line: *"good suggestions when confident, safe rejection when uncertain — that's production AI."*

Study these FlyRank parts first

FlyRank reference	What to steal from it
<code>inngest/generateImageForContent.ts</code> · <code>lib/dynamic-image-variants/</code>	Image generation + variants — context for where your engine plugs in
<code>config/agent-flows.config.ts</code>	Agent-based image quality checks — the idea your guard systematizes
<code>inngest/blog-existence-check/checkBlogExistenceBatch.ts</code>	The batch-classification pattern to model your vision job on
<code>lib/agents/schema/builder.ts</code> · <code>config/chat-pricing.config.ts</code>	Structured output + AI cost tracking, done the FlyRank way

14 · Curated resources — free, verified, leveled

Don't read everything. Each row says when to reach for it. Every link was verified while writing this brief; every resource is free with no credit card. If a link ever dies, the title is searchable.

● **Start here** (everyone) · ● **When you're comfortable** · ● **Optional deep dive** | Lanes: **JS** · **PY** · **both**

Phase 1 · Design

Lane	Resource	Format	When to use it
● both	Google ML Crash Course — Embeddings	Interactive, ~45 min	Before designing the shared semantic space — why captions and post text become comparable vectors.
● both	Hugging Face — Vision language models explained	Article, ~12 min	When choosing your vision model — how image encoder + text decoder fit together.
● both	JSON Schema — Getting started	Tutorial, ~20 min	While designing the tag schema: types, required fields, constraints.
● both	Unsplash License · Pexels License	2 min each	Before collecting the ~50-image corpus — confirm free use.

Phase 2 · Vision pipeline

Lane	Resource	Format	When to use it
● both	Gemini API — Image understanding	Docs, ~15 min	Core reference for sending images to Gemini Flash; Python + JS examples.
● both	Gemini API — Structured output	Docs, ~15 min	To get schema-conforming JSON tags instead of prose — Pydantic and Zod wired directly in.
● both	Ollama — Vision models	Docs, ~10 min	The \$0 local alternative: vision models with no API key, offline.
● JS	Zod — Basics	Docs, ~10 min	Define the tag schema once; <code>safeParse()</code> every model response.
● PY	Pydantic — Models	Docs, ~15 min	<code>model_validate_json()</code> to reject malformed tag output.

Phase 3 · Matching & the guard

Lane	Resource	Format	When to use it
● both	Gemini API — Embeddings	Docs, ~10 min	Embed captions + posts into one space: SEMANTIC_SIMILARITY task type + cosine example.
● PY	Build an AI image tagger (Llama Vision + ChromaDB)	Build-along, ~1 h	The closest free build-along: local vision tagging → vectors → semantic search.

Phase 4 · Eval & hardening

Lane	Resource	Format	When to use it
------	----------	--------	----------------

●	both	Google ML Crash Course — Precision & recall	Interactive, ~20 min	When building the labeled eval set — exactly what your top-1 precision measures.
●	both	Gemini API — Pricing & free tier	Reference, ~5 min	For per-call cost tracking and staying inside the free tier.

15 · Glossary

Plain-language definitions of the **bold** terms in this brief. No definition depends on another — read in any order.

Term	What it means
Vision model	An AI model that takes an image (plus a prompt) and produces text about it — here, structured tags and captions.
Structured output	Forcing a model to answer in a machine-checkable shape (JSON matching a schema) instead of free prose.
Schema validation	Automatically checking that data has the right fields and types before your code trusts it — Zod/Pydantic here.
Confidence score	The model's own 0–1 estimate of how sure it is. Low confidence → flag for review, never silently accept.
Embedding	A list of numbers representing meaning, so "red fox" and "Vulpes vulpes" land close together despite different words.
Vector index	Storage optimized for "find the closest vectors to this one" — how ranking candidates are fetched.
Cosine similarity	A score for how close two embeddings point in the same direction — your ranking metric.
Similarity threshold	The tuned cut-off below which a match is refused. Set with your eval data, not guessed.
Mismatch guard	The safety layer combining tags + thresholds + confidence to reject wrong pairings with an explanation.
Batch job	Processing many items in the background with retries and progress — how all vision/embedding calls run.
Cost tracking	Recording what every AI call costs, attributed per call — a habit that matters even at \$0.
Eval set	A small hand-labeled dataset ("this post's correct image is X") used to measure quality objectively.
Top-1 precision	Of all posts, the share whose <i>first</i> suggested image was the labeled correct one. Your headline quality number.

FlyRank Internship · Backend Development Track · Capstone — AI Image Understanding & Content Matching Engine. Everything in this brief can be completed with free tools; no resource linked here requires a credit card. Questions → the capstone channel on the community. Version: July 2026.